I-Logical Approaches Propositional Syntax and Tensor Network Decompositions

Foundations of Neuro-Symbolic AI

Alex Goessmann

University of Applied Science Würzburg-Schweinfurt (THWS)

April 13, 2026

Outline

- Propositional syntax
- Basis encodings for basis calculus
- Decompositions and message passing inference
- Rules of replacement and clauses
- Connection to knowledge base representations
- Implementations in `tnreason`

Motivation: Sparsity in formulas

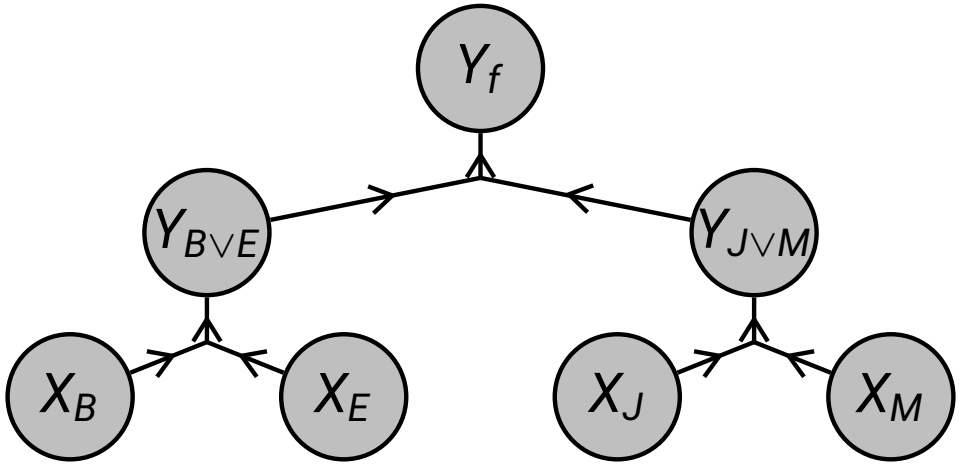
Recall: Tensor networks as a sparsity principle

- Knowledge base decomposition into formulas
- Can we exploit further sparsity in the syntax of the formulas?

Example: The formula *"When a burglary or an earthquake occurs, either our neighbor John or Mary calls."* has a representation by

$$f\,[X_B, X_E, X_J, X_M] = (X_B \vee X_E) \Rightarrow (X_J \vee X_M)$$

Intuitively we can decompose the formula as

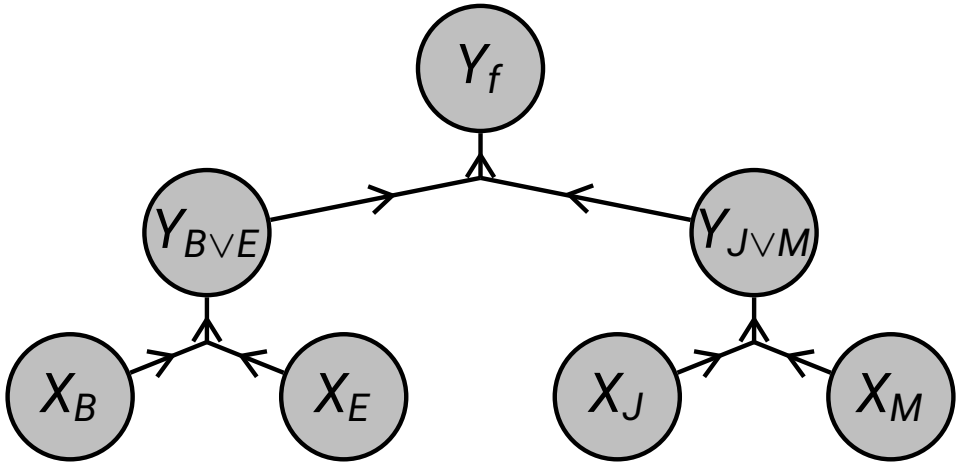


Motivation: Sparsity in formulas

Example: The formula *"When a burglary or an earthquake occurs, either our neighbor John or Mary calls."* has a representation by

$$f\,[X_B, X_E, X_J, X_M] = (X_B \vee X_E) \Rightarrow (X_J \vee X_M)$$

Intuitively we can decompose the formula as



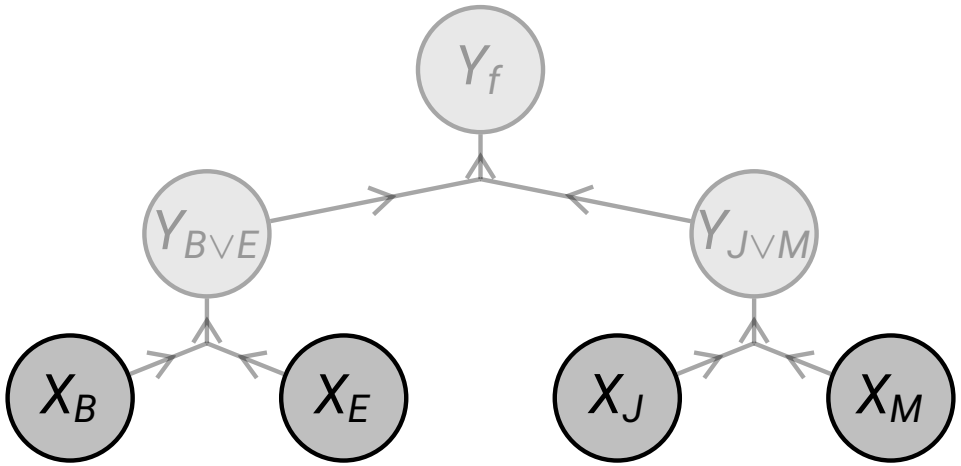
In this lecture we formalize this intuition:

- understand each Y as a hidden variable
- decorate directed hyperedges by basis encodings to find tensor network decompositions

Propositional Syntax

Propositional syntax (e.g. $f = (X_B \vee X_E) \Rightarrow (X_J \vee X_M)$) consists of

- Atomic formulas (here: X_B, X_E, X_J, X_M)
- Connectives (here: $\vee, \Rightarrow$)

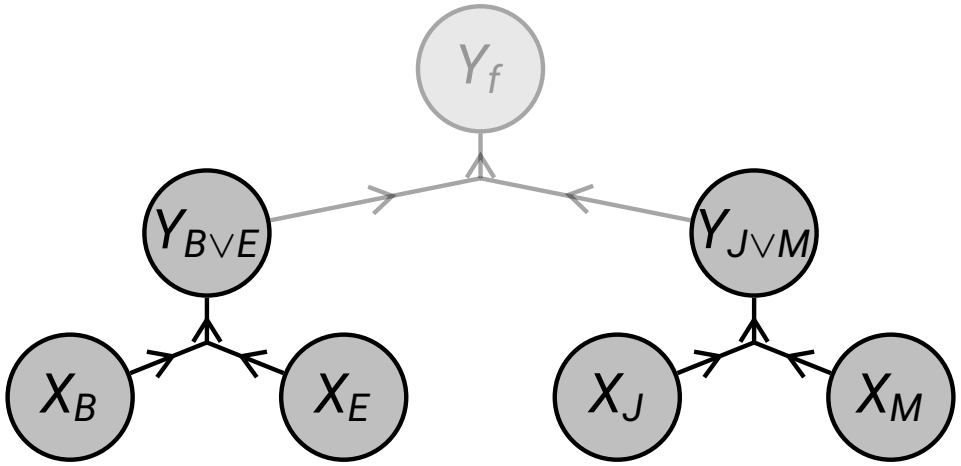


Atomic formulas ("propositional symbols"), e.g. X_B:
A world $(x_B, x_E, x_J, x_M) \in [2]^4$ is a model, if and only if $x_B = 1$.

Propositional Syntax

Propositional syntax (e.g. $f = (X_B \vee X_E) \Rightarrow (X_J \vee X_M)$) consists of

- Atomic formulas (here: X_B, X_E, X_J, X_M)
- Connectives (here: $\vee, \Rightarrow$)



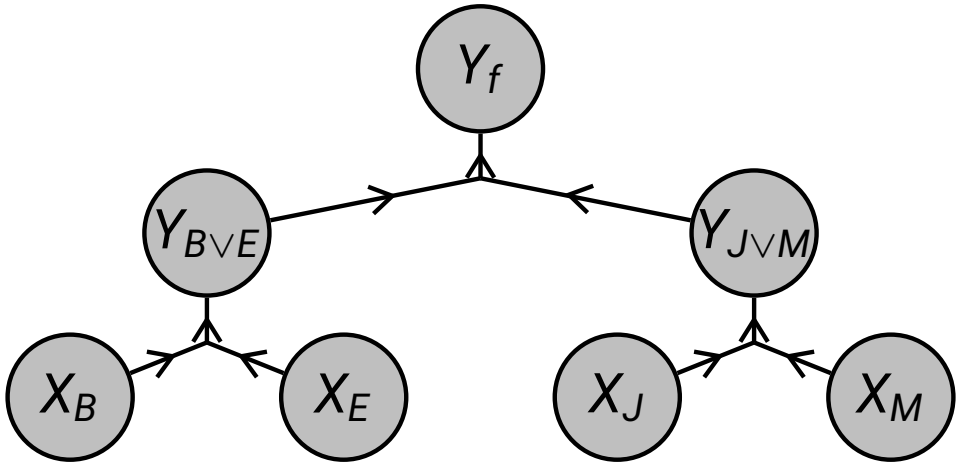
Connectives building **complex formulas**, e.g. $X_B \vee X_E$:
Satisfaction computed by the satisfaction of the sub-formulas

X_B	X_E	$X_B \vee X_E$
0	0	0
0	1	1
1	0	1
1	1	1

Propositional Syntax

Propositional syntax (e.g. $f = (X_B \vee X_E) \Rightarrow (X_J \vee X_M)$) consists of

- Atomic formulas (here: X_B, X_E, X_J, X_M)
- Connectives (here: $\vee, \Rightarrow$)



Connectives building **complex formulas**, e.g. $X_B \vee X_E$:
Satisfaction computed by the satisfaction of the sub-formulas

$X_B \vee X_E$	$X_J \vee X_M$	f
0	0	1
0	1	1
1	0	0
1	1	1

How to represent formulas as tensor networks?

Task: Exploit the syntax

The syntactical representation of formulas is the sparsity principle of propositional logics to avoid the curse of dimensions. Can we capture this principle by tensor network decompositions?

To be more precise:

- Suppose we have a tensor network decomposition of formulas f_0 and f_1.
- How to contract the tensor networks to get $f_0 \circ f_1$ (where $\circ \in \{\vee, \wedge, \Rightarrow, \cdots\}$)?

Recall: Contraction of both formulas (or tensor network decomposition of them) is the logical conjunction

$$\left(f_0 \wedge f_1\right)[X_{[d]}] = \left\langle f_0\left[X_{[d]}\right], f_1\left[X_{[d]}\right]\right\rangle_{[X_{[d]}]}$$

The non-linearity problem

Multilinearity of tensor networks

Tensor networks are multilinear, that is linear in each core tensor decorating an edge.

The logical conjunction is the only connective which is multilinear.

Non-linearity problem

How to represent non-linear formulas like $\neg X$ in a tensor network?

Solution:
We develop the basis calculus scheme instead of coordinate calculus.

Outline

- Propositional syntax
- Basis encodings for basis calculus
- Decompositions and message passing inference
- Rules of replacement and clauses
- Connection to knowledge base representations
- Implementations in `tnreason`

Recall: One-hot encodings

One-hot encodings of a state are a tensor with a single coordinate 1 marking the corresponding state.

One-hot encodings

For a state $x \in [m]$ of a variable X the one-hot encoding is

$$\boxed{\epsilon_{x_0}} \quad = \begin{pmatrix} 0 & \cdots & 0 & 1 & 0 & \cdots & 0 \end{pmatrix} [X_0]$$

$$\begin{array}{ccccccc} \vdots & & & \vdots & & & \vdots \\ 0 & & & x_0 & & & m_0 - 1 \end{array}$$

Given a factored representation with multiple variables $X_{[d]}$, the one-hot encoding of a state $x_{[d]}$ is the tensor product

$$\epsilon_{x_{[d]}} \left[X_{[d]}\right] = \bigotimes_{k \in [d]} \epsilon_{x_k} \left[X_k\right] . \tag{1}$$

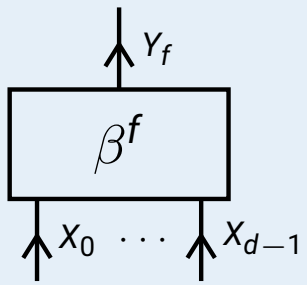
They build an orthogonal basis of the tensor space.

Basis encodings

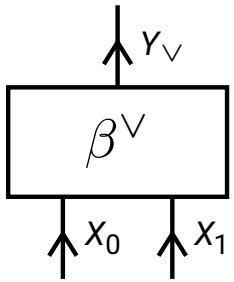
Definition (Basis encoding)

The basis encodings of a formula $f\left[X_{[d]}\right] : \{0,1\}^d \to \{0,1\}$ is the tensor

$$\beta^f\left[Y_f, X_{[d]}\right] = \sum_{x_{[d]} \in [2]^d} \epsilon_{f(x_{[d]})}\left[Y_f\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right] \quad ,$$



Example: The basis encoding of the logical disjunction $\vee$ is the tensor



$$= \quad \begin{array}{c} X_1 \\ \begin{array}{cc} 0 & 1 \end{array} \\ X_0 \begin{array}{c} 0 \\ 1 \end{array} \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \\ \begin{array}{cc} 0 & 1 \\ & Y_\vee \end{array} \end{array} \quad [X_0, X_1, Y_\vee]$$

Coordinate and basis encoding

Coordinate encoding:
Store an information $f(x_{[d]})$ in the coordinates of a tensor

$$f\left[X_{[d]}\right] = \sum_{x_{[d]} \in [2]^d} f(x_{[d]}) \cdot \epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

Basis encoding:
Store an information $f(x_{[d]})$ in a basis vector $\epsilon_{f(x_{[d]})}\left[Y_f\right]$ to an auxiliary variable Y_f, that is

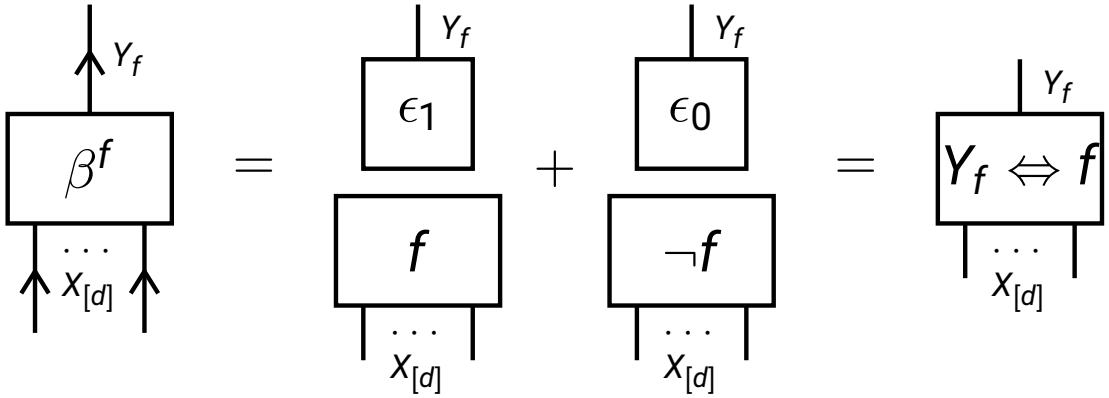
$$\beta^f\left[Y_f, X_{[d]}\right] = \sum_{x_{[d]} \in [2]^d} \epsilon_{f(x_{[d]})}\left[X_{[d]}\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

Hidden variables perspective

Basis encodings introduce hidden variables Y_f representing the satisfaction of the formula

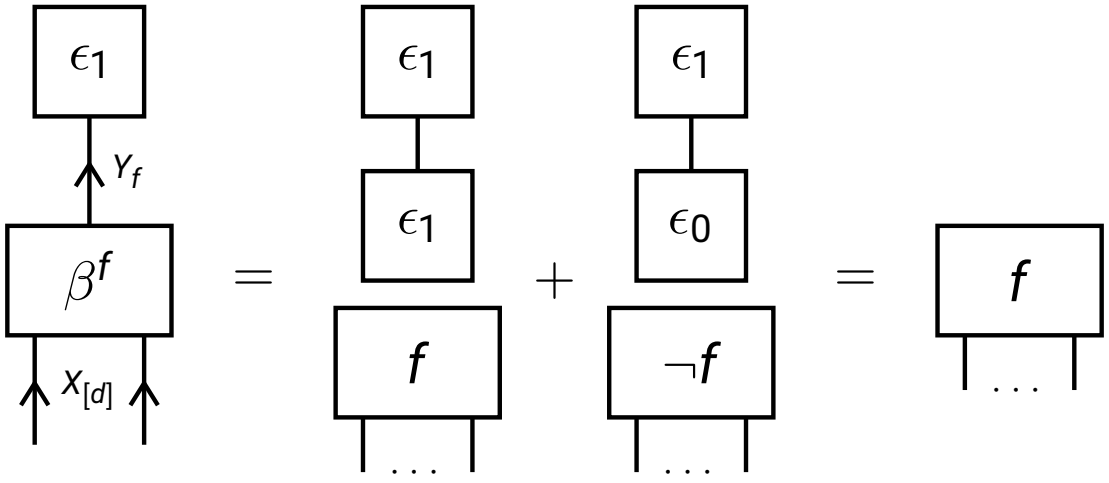
$$\beta^f \left[Y_f, X_{[d]}\right] = (Y_f \Leftrightarrow f) \left[Y_f, X_{[d]}\right]$$

We depict this as



Retrieval of the formula

We retrieve the formula $f\left[X_{[d]}\right]$ from its basis encoding by contracting with $\epsilon_1\left[Y_f\right]$:



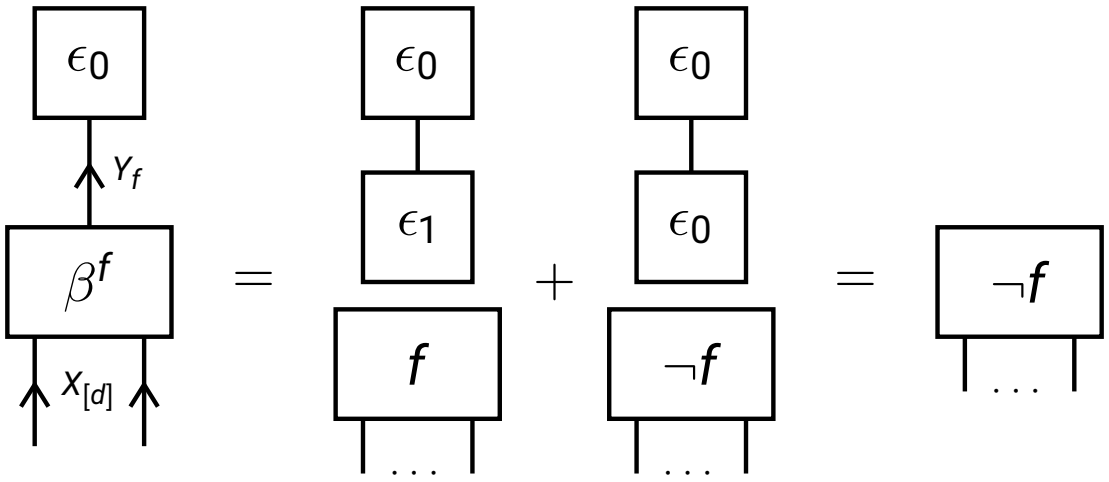
Solution of the non-linearity problem

Basis encodings resolve the nonlinearity problem: Any composition of formulas can be represented as a tensor network.

Example: Logical negation of a formula

$$\neg f\left[X_{[d]}\right] = \left\langle \beta^f\left[Y_f, X_{[d]}\right], \epsilon_0\left[Y_f\right]\right\rangle_{\left[X_{[d]}\right]}$$

by



Basis calculus perspective on logics

Vector representation of Booleans

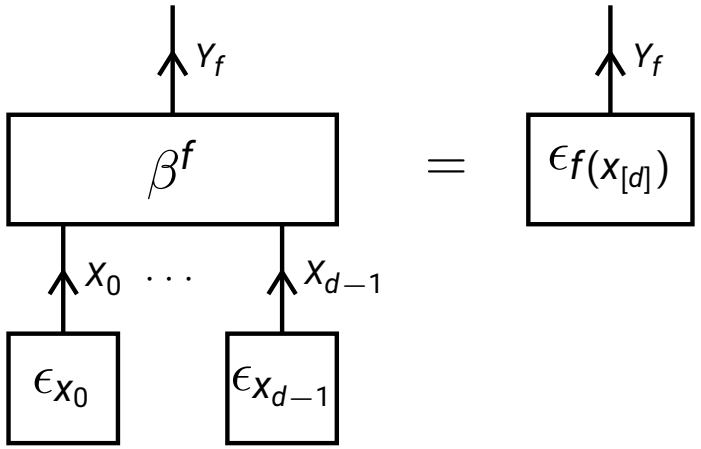
Boolean states can be represented by one-hot encodings

$$\mathrm{True} \leftrightarrow \begin{pmatrix} 0 \\ 1 \end{pmatrix} [X] \quad \text{and} \quad \mathrm{False} \leftrightarrow \begin{pmatrix} 1 \\ 0 \end{pmatrix} [X] \,.$$

Function evaluation by contraction: For each $x_{[d]} \in [2]^d$ we have

$$\epsilon_{f(x_{[d]})} [Y_f] = \left\langle \beta^f \left[Y_f, X_{[d]} \right], \epsilon_{x_{[d]}} \left[X_{[d]} \right] \right\rangle_{[Y_f]} \,.$$

That is:



Basis calculus

Idea of basis calculus:

- Represent input $x_{[d]}$ by one-hot encoding $\epsilon_{x_{[d]}}\left[X_{[d]}\right]$
- Contract with the basis encoding of a function
- Get one-hot encoding of $f(x_{[d]})$ as a result

Basis calculus scheme

States are represented by basis vectors (their one-hot encodings) and processed by contractions with basis encodings.

Outline

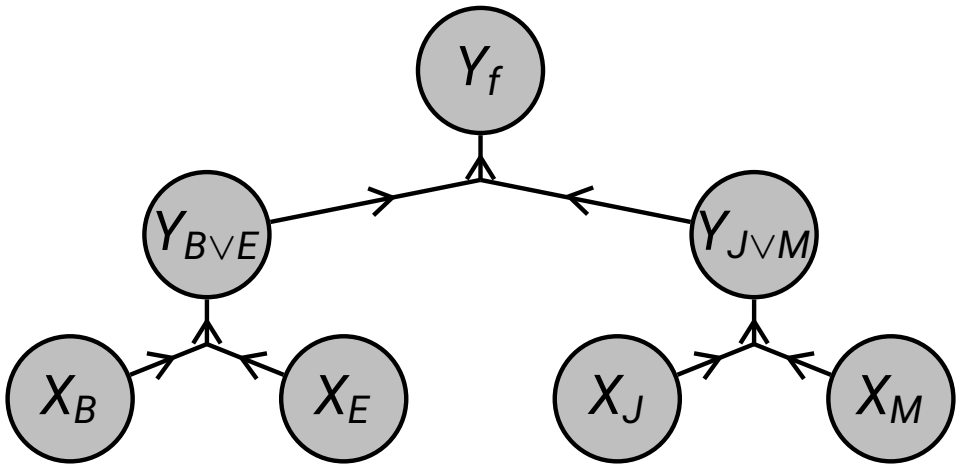
- Propositional syntax
- Basis encodings for basis calculus
- Decompositions and message passing inference
- Rules of replacement and clauses
- Connection to knowledge base representations
- Implementations in `tnreason`

Decomposition of basis encodings

Recall our motivation:

- Formulas are high-dimensional tensors, but they (often) have structure by their syntax
- Propositional syntax introduces multiple hidden variables
- Use syntax to find tensor network decompositions

Example: $f\,[X_B, X_E, X_J, X_M] = (X_B \vee X_E) \Rightarrow (X_J \vee X_M)$

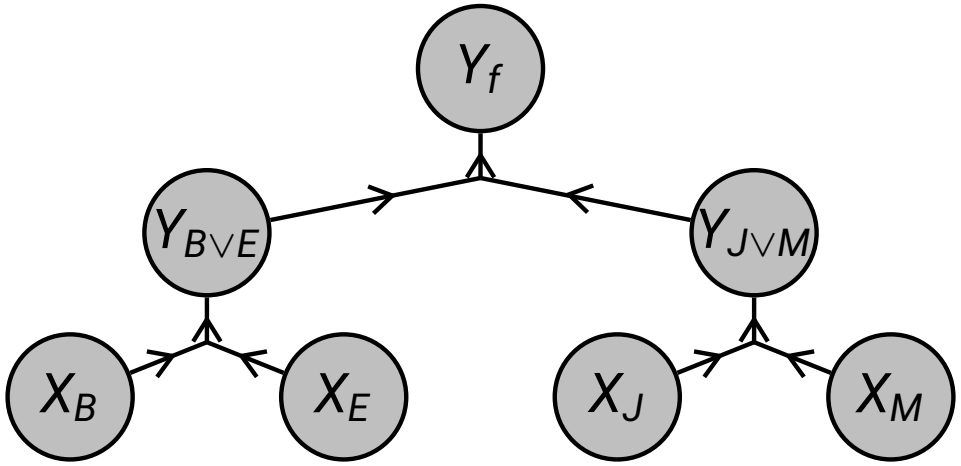


Idea:

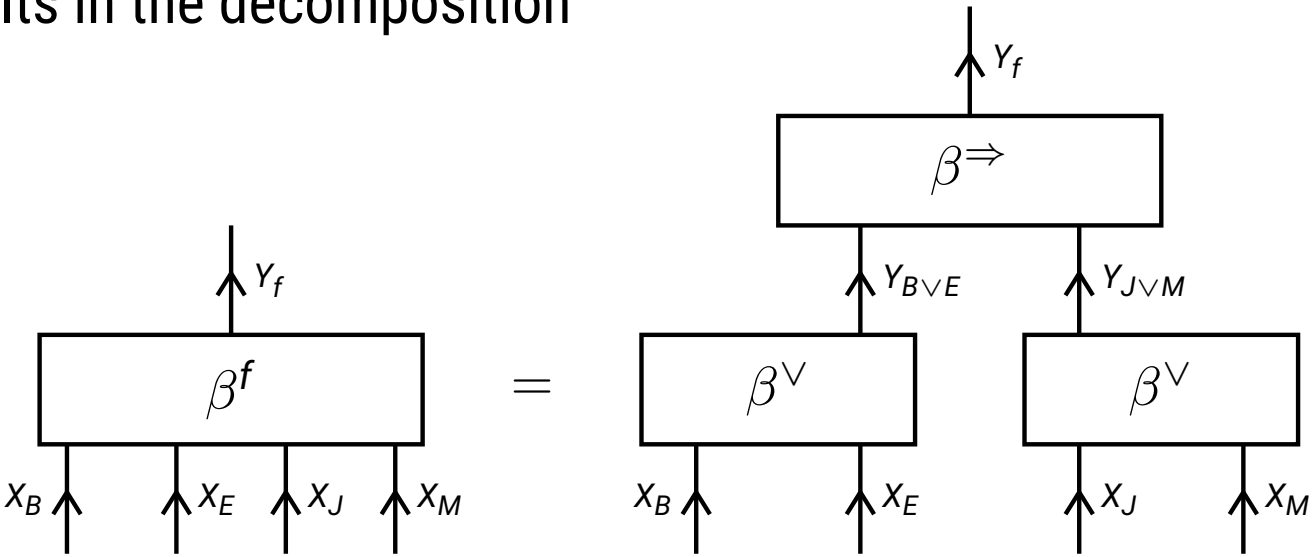
- Introduce for each sub-formula a hidden variable Y
- Prepare a basis encoding for each sub-formula

Decomposition of basis encodings

Example: $f\,[X_B, X_E, X_J, X_M] = (X_B \vee X_E) \Rightarrow (X_J \vee X_M)$



This results in the decomposition



Decomposition sparsity

Theorem

The basis encoding of a composed formula is the contraction of the basis encodings to its connectives.

Proof sketch: When contracting the decomposition with one-hot encodings of any $x_{[d]} \in [2]^d$, we get by basis calculus the one-hot encodings of the composition formulas, and finally the one-hot encoding of the whole formula. This coincides with contracting the full basis encoding since

$$\left\langle \beta^f \left[Y_f, X_{[d]}\right], \epsilon_{x_{[d]}} \left[X_{[d]}\right] \right\rangle_{[Y_f]} = \epsilon_{f(x_{[d]})} \left[Y_f\right] .$$

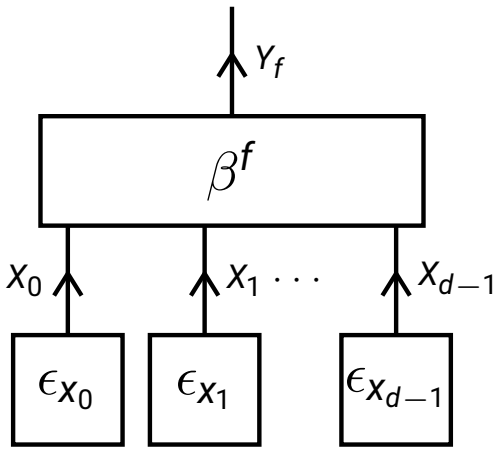
Decomposition sparsity

When a function can be decomposed into connective functions, its corresponding tensor has a tensor network decomposition by the basis encodings of the connective functions.

Decomposition sparsity for inference

Recall: Logical inference (deciding entailment) requires tensor network contractions. Since naive computation schemes are intractable, can we exploit tensor network decompositions for efficient inference?

Formula evaluation as an inference instance:



In the entailment perspective:

- Knowledge by a state: $\mathcal{KB}\left[X_{[d]}\right] = \epsilon_{X_{[d]}}\left[X_{[d]}\right]$
- Investigate whether formula f is entailed amounts to deciding whether $f(x_{[d]}) = 1$

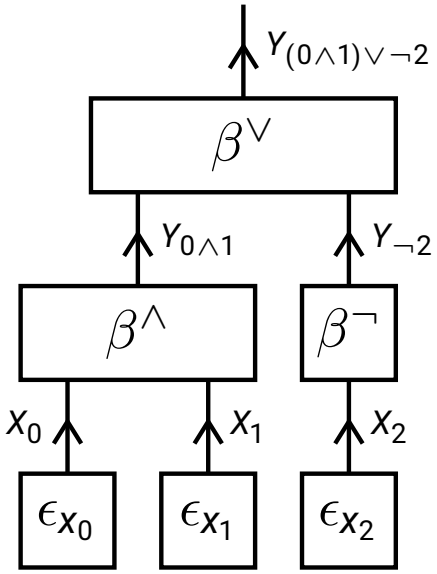
Formula evaluation by message passing of one-hot encodings

Basis Calculus: Computation by basis vectors

- Input states are prepared by their one-hot encodings
- Contractions with the basis encodings result in the one-hot encoding of the output

Advantage: Decomposition of basis encodings enables to decompose global contractions into local contractions.

Example: Evaluation of the formula $f\left[X_{[3]}\right] = \left((X_0 \wedge X_1) \vee \neg X_2\right)$



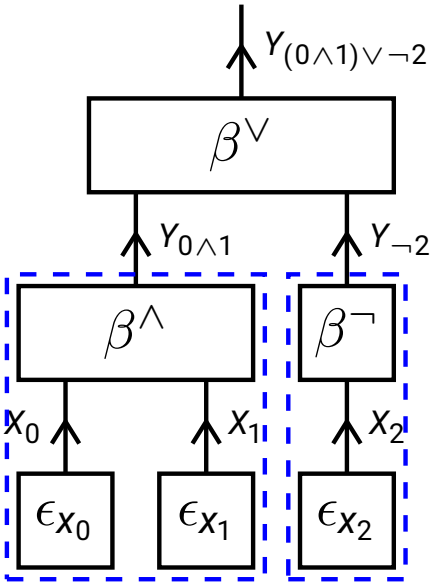
Formula evaluation by message passing of one-hot encodings

Basis Calculus: Computation by basis vectors

- Input states are prepared by their one-hot encodings
- Contractions with the basis encodings result in the one-hot encoding of the output

Advantage: Decomposition of basis encodings enables to decompose global contractions into local contractions.

Example: Evaluation of the formula $f\left[X_{[3]}\right] = \left((X_0 \wedge X_1) \vee \neg X_2\right)$



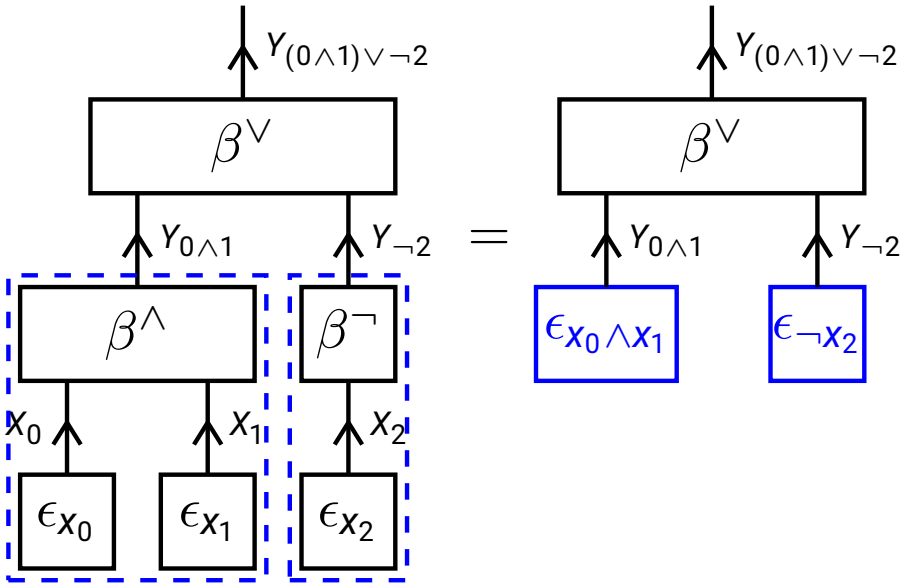
Formula evaluation by message passing of one-hot encodings

Basis Calculus: Computation by basis vectors

- Input states are prepared by their one-hot encodings
- Contractions with the basis encodings result in the one-hot encoding of the output

Advantage: Decomposition of basis encodings enables to decompose global contractions into local contractions.

Example: Evaluation of the formula $f\left[X_{[3]}\right] = \left((X_0 \wedge X_1) \vee \neg X_2\right)$



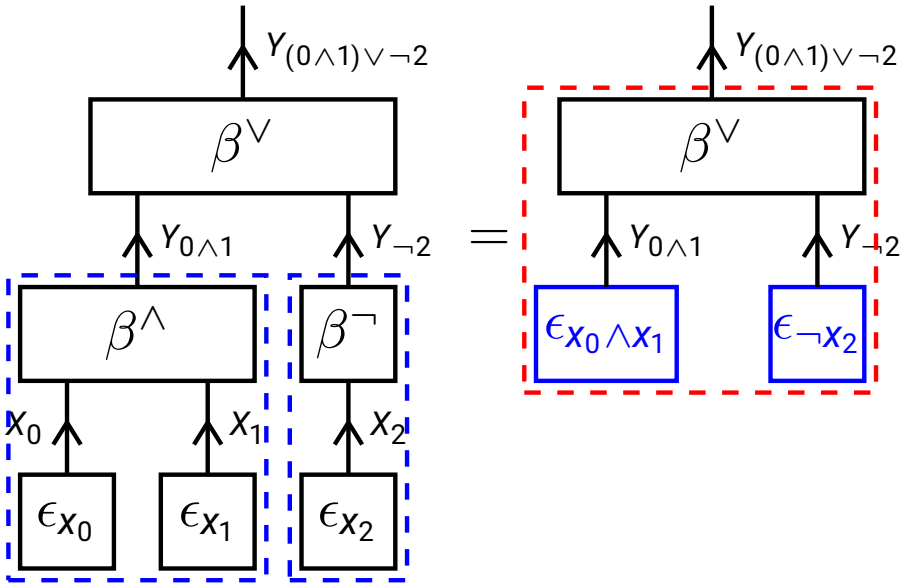
Formula evaluation by message passing of one-hot encodings

Basis Calculus: Computation by basis vectors

- Input states are prepared by their one-hot encodings
- Contractions with the basis encodings result in the one-hot encoding of the output

Advantage: Decomposition of basis encodings enables to decompose global contractions into local contractions.

Example: Evaluation of the formula $f\left[X_{[3]}\right] = \left((X_0 \wedge X_1) \vee \neg X_2\right)$



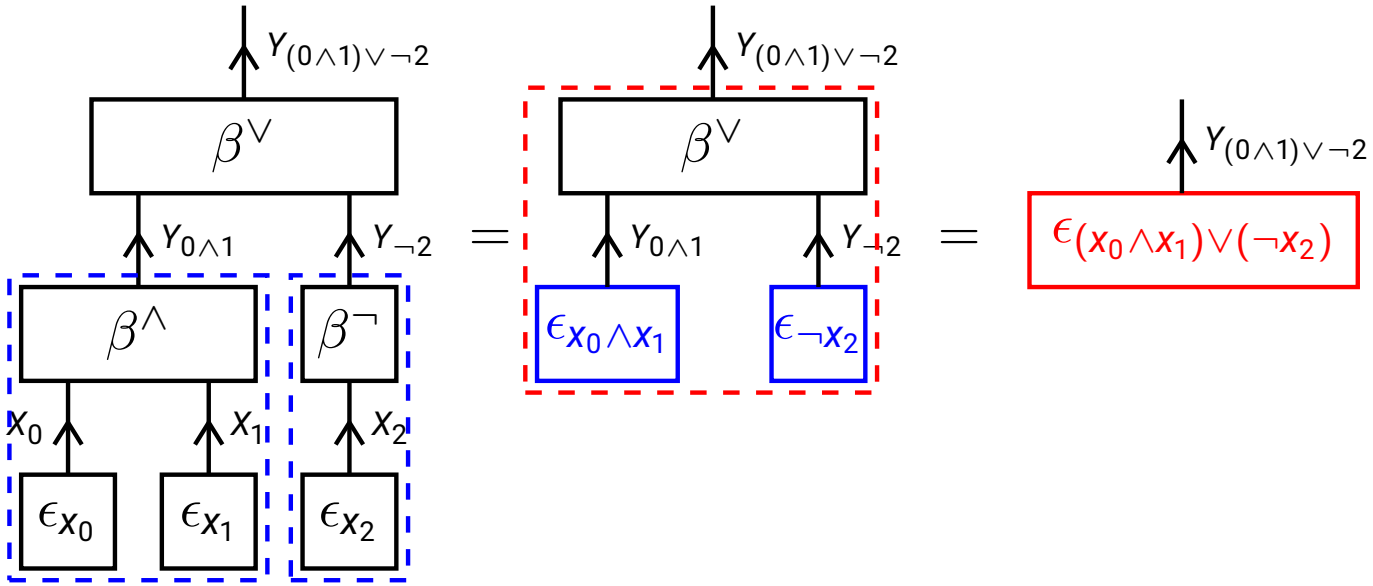
Formula evaluation by message passing of one-hot encodings

Basis Calculus: Computation by basis vectors

- Input states are prepared by their one-hot encodings
- Contractions with the basis encodings result in the one-hot encoding of the output

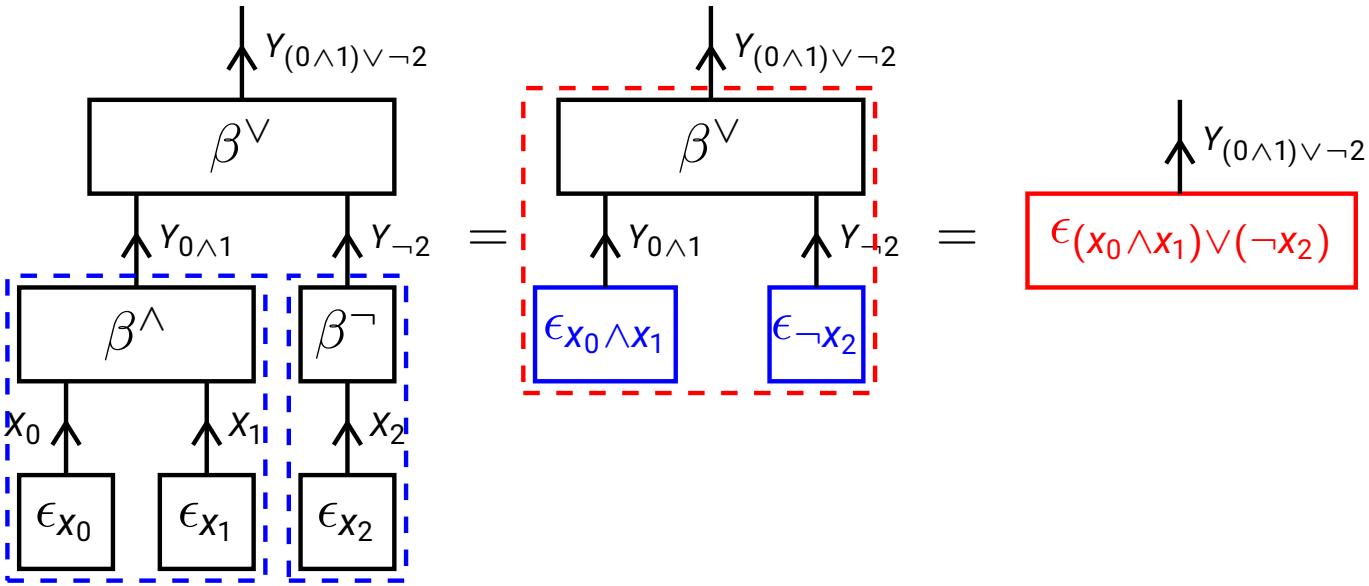
Advantage: Decomposition of basis encodings enables to decompose global contractions into local contractions.

Example: Evaluation of the formula $f\left[X_{[3]}\right] = \left((X_0 \wedge X_1) \vee \neg X_2\right)$



Formula evaluation by message passing of one-hot encodings

Example: Evaluation of the formula $f\left[X_{[3]}\right] = \Big((X_0 \wedge X_1) \vee \neg X_2\Big)$



This is a first example of a message passing scheme:

- **Messages** are computed by contractions of sub-networks (two blue contractions, one red contraction)
- **Passing** of previous messages (blue contractions) to compute new messages (red contraction)

Outline

- Propositional syntax
- Basis encodings for basis calculus
- Decompositions and message passing inference
- Rules of replacement and clauses
- Connection to knowledge base representations
- Implementations in `tnreason`

Manipulations of syntax

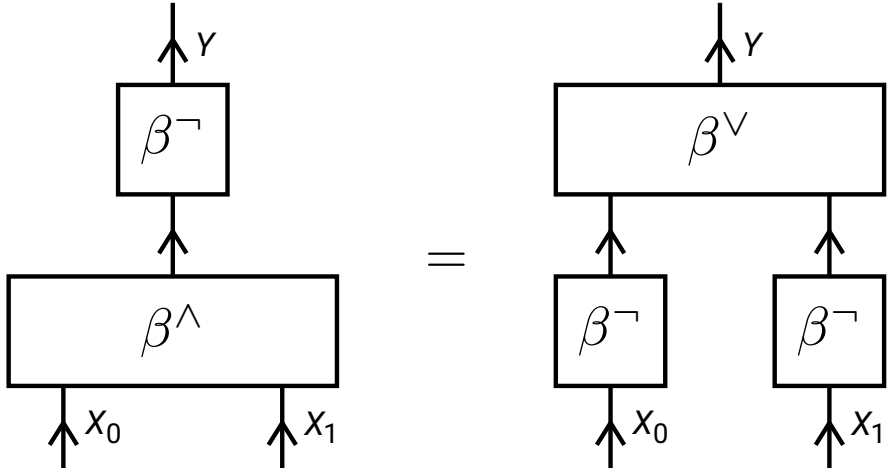
There are different syntactical representations of the same formula, some of them are found by syntactical manipulations.

Example: De Morgan rule $\neg\left(X_0 \wedge X_1\right) \equiv \left(\neg X_0 \vee \neg X_1)\right)$

Tensor network perspective

Equivalence of syntactical representations correspond with equivalence of corresponding tensor network contractions.

The De Morgan rule corresponds with the contraction equation:



Rules of replacement

For any propositional formulas f_0 and f_1 the following is true:

- Elimination of bi-implications

$$\Big(f_0 \Leftrightarrow f_1\Big) \equiv \Big((f_0 \Rightarrow f_1) \wedge (f_1 \Rightarrow f_0)\Big)$$

- Elimination of implications

$$\Big(f_0 \Rightarrow f_1\Big) \equiv \Big(\neg f_0 \vee f_1)$$

- De Morgan rules

$$\neg\Big(f_0 \wedge f_1\Big) \equiv \Big(\neg f_0 \vee \neg f_1\Big) \quad ,$$
$$\neg\Big(f_0 \vee f_1\Big) \equiv \Big(\neg f_0 \wedge \neg f_1\Big)$$

- Distributive rule

$$\Big(f_0 \vee (f_1 \wedge f_2)\Big) \equiv \Big((f_0 \vee f_1) \wedge (f_0 \vee f_2)\Big)$$

Clauses and conjunctive normal form

- Atomic formulas and their negations are called **positive and negative literals**. For example:

$$\neg X_B \quad \text{is a negative literal} \quad , \quad X_J \quad \text{is a positive literal}$$

- **Clauses** are disjunctions of literals. For example:

$$\neg X_B \vee X_J \vee X_M$$

- A syntactical representation of a formula is in the **conjunctive normal form**, if it is a conjunction of clauses (a knowledge base of clauses).

Parametrization of clauses

We parametrize a clause by

- a set $\mathcal{U}$ marking the atomic formulas appearing in the clause
- indices $y_{\mathcal{U}} \in [2]^{|\mathcal{U}|}$ storing whether the literal is positive or negative

$$y_k = \begin{cases} 0 & \text{If the literal is positive, i.e. } X_k \text{ appearing in the clause} \\ 1 & \text{If the literal is negative, i.e. } \neg X_k \text{ appearing in the clause} \end{cases}$$

and denote it by $Z^{\vee}_{\mathcal{U},y_{\mathcal{U}}}$.

The tensor to the clause is decomposed as:

$$Z^{\vee}_{\mathcal{U},y_{\mathcal{U}}} = \underbrace{\mathbb{I}\left[X_{\mathcal{U}}\right]}_{\text{always satisfied}} - \underbrace{\bigotimes_{k\in\mathcal{U}} \epsilon_{y_k}\left[X_k\right]}_{\text{except for the world } y_{\mathcal{U}}}$$

We exploit this structure for efficient storage and processing of clauses.

Conjunctive normal form (CNF)

Using the rules of replacement, any syntactical representation of a formula (involving only unary and binary connectives) can be transfered into the CNF.

Example: The formula $f = (X_B \vee X_E) \Rightarrow (X_J \vee X_M)$ has a CNF with the clauses $(\neg X_B \vee X_B \vee X_E)$ and $(\neg X_E \vee X_B \vee X_E)$. We find the CNF by the rules of equivalence:

$$
\begin{aligned}
&(X_B \vee X_E) \Rightarrow (X_J \vee X_M) && \text{(eliminate the implication)}\\
&\equiv \neg (X_B \vee X_E) \vee (X_J \vee X_M) && \text{(apply De Morgan)}\\
&\equiv (\neg X_B \wedge \neg X_E) \vee (X_J \vee X_M) && \text{(apply the distributive law)}\\
&\equiv \underbrace{(\neg X_B \vee X_J \vee X_M)}_{\text{first clause}} \wedge \underbrace{(\neg X_E \vee X_J \vee X_M)}_{\text{second clause}}
\end{aligned}
$$

Outline

- Propositional syntax
- Basis encodings for basis calculus
- Decompositions and message passing inference
- Rules of replacement and clauses
- Connection to knowledge base representations
- Implementations in `tnreason`

Representation of knowledge bases

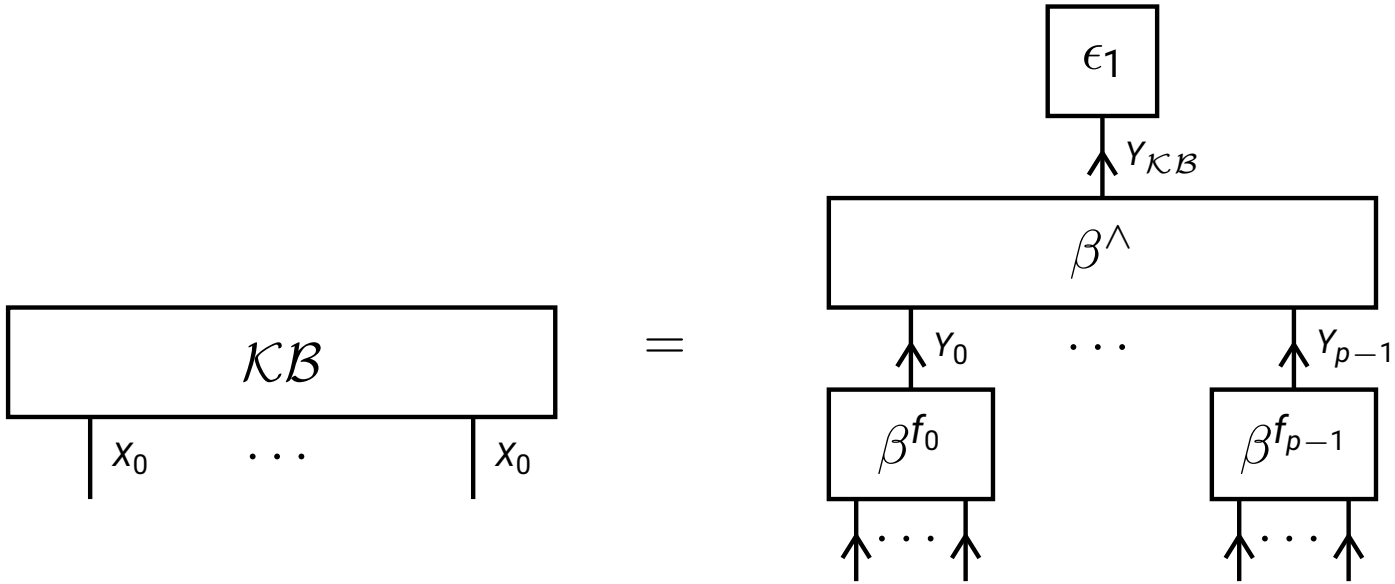
Recall: A knowledge base is the co

$$\mathcal{KB}\left[X_{\mathcal{V}}\right] = \left\langle \left\{ f_\ell \left[X_{\mathsf{e}_\ell}\right] \,:\, \ell \in [p] \right\} \right\rangle_{[X_{\mathcal{V}}]} \,.$$

The knowledge base is a formula itself

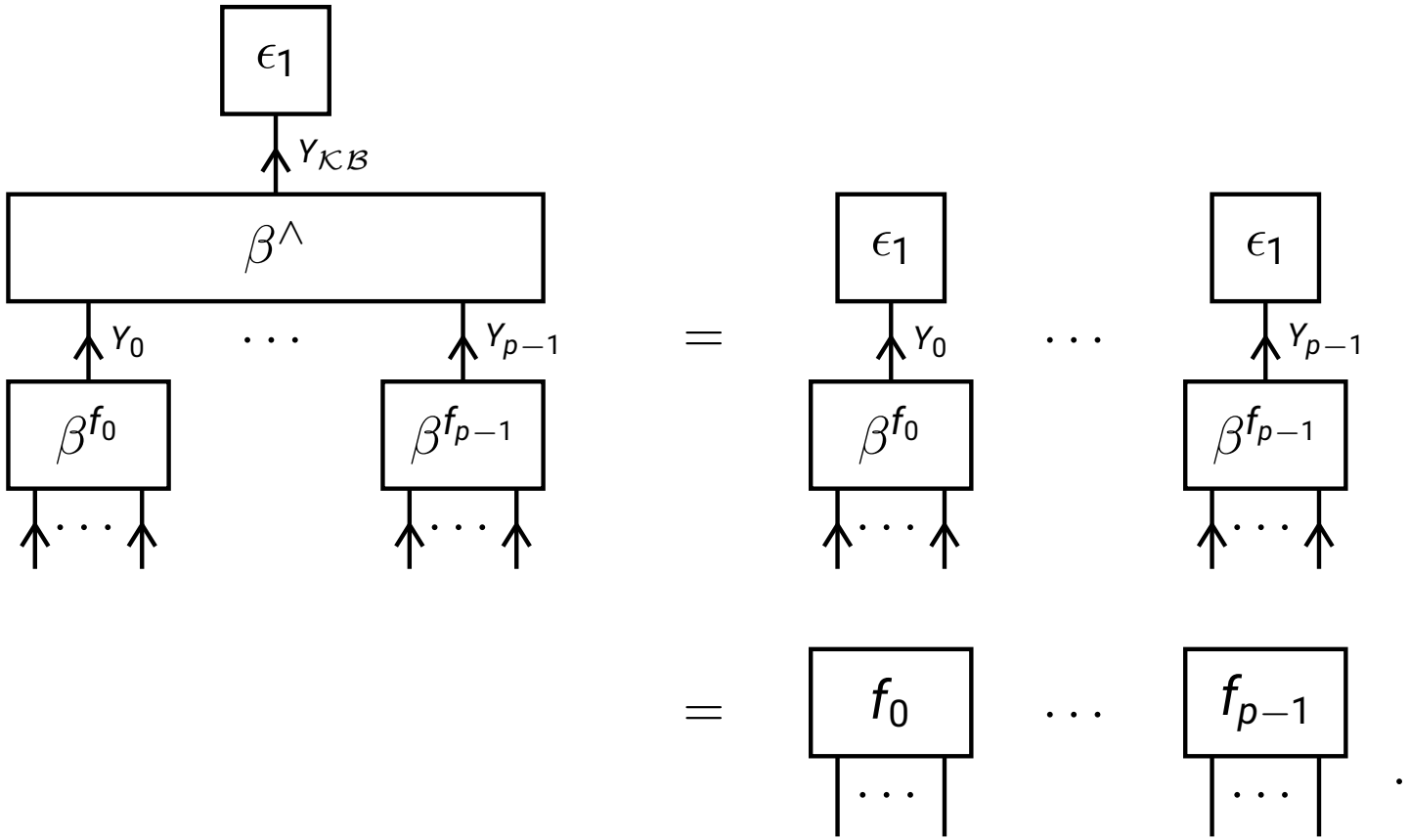
$$\mathcal{KB} = \bigwedge_{\ell \in [p]} f_\ell$$

and has a representation by decomposed basis encodings:



Representation of knowledge bases

Both representations are equal since



We simplified the basis encoding of the conjunction by

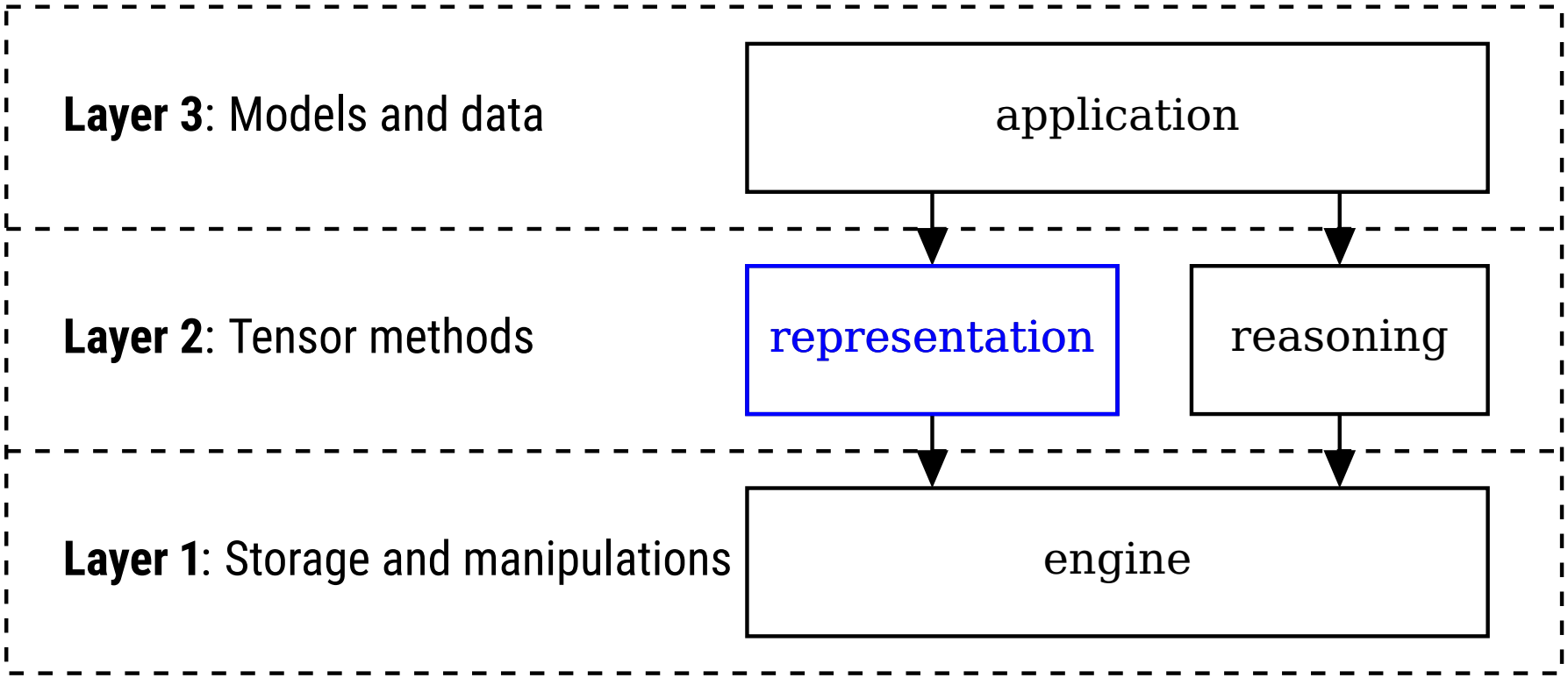
$$\beta^\wedge \left[Y_\wedge = 1, X_{[d]}\right] = \bigotimes_{k \in [d]} \epsilon_1 \left[X_k\right] .$$

Outline

- Propositional syntax
- Basis encodings for basis calculus
- Decompositions and message passing inference
- Rules of replacement and clauses
- Connection to knowledge base representations
- Implementations in `tnreason`

Implementations in tnreason**:** **Subpackage** representation

The subpackage representation is dedicated to the tensor encoding schemes such as basis encodings.



Subpackage representation**: Basis encodings**

Creation of a basis encoding $\beta^f\left[Y_f, X_{[d]}\right]$:

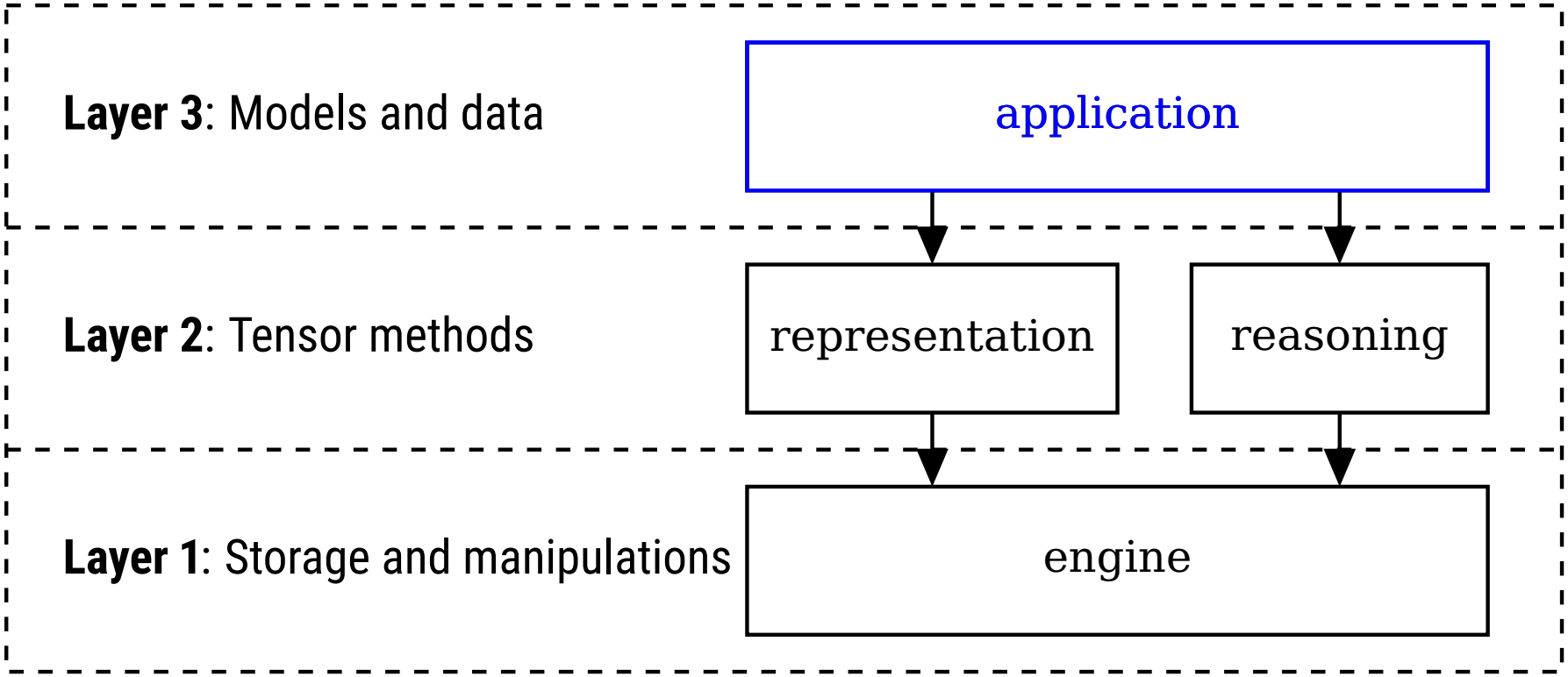
```
from tnreason import representation
basisEncodedCore =
↪  representation.create_basis_encoding_from_lambda(
   inShape, outShape, inColors, outColors, function)
```

- inShape, outShape, inColors, outColors: specifying the leg dimension and the names of the incoming variables ($X_{[d]}$) and the outgoing variables (Y_f)
- function: computes the state of the outgoing variables given the state of the incoming variables

Implementations in tnreason: Subpackage application

The subpackage application provides high-level models

- Interpretation of propositional syntax
- Training on empirical data (later)



Representation of Syntax

In tnreason, propositional syntax is represented by nested lists of strings, for example:

```
["and", ["not", "Rained"], ["imp", "Rained", "Wet"]]
```

Connectives $\circ$ get assigned a string $s(\circ)$:

Unary connective $\circ$	$s(\circ)$
$\neg$	*"not"*
$()$	*"id"*

Binary connective $\circ$	$s(\circ)$
$\wedge$	*"and"*
$\vee$	*"or"*
$\Rightarrow$	*"imp"*
$\oplus$	*"xor"*
$\Leftrightarrow$	*"eq"*

Any other string is interpreted by an atomic formula (a so-called propositional symbol).

Propositional syntax in tnreason

Having strings representing

- Connectives (by defined representations)
- Atomic Formulas (all other strings)

we represent formulas $f_1 \circ f_2$

$$s(f_1 \circ f_2) = [s(\circ), s(f_1), s(f_2)]$$

where we apply the conventions

- Connectives are at the 0th position in each list
- Further entries are either atoms as strings or encoded formulas itself

Examples of representations

Atomic variable Rained by

$s(\text{Rained})$ = `"Rained"`

Negative literal $\neg$Rained by

$s(\neg\text{Rained})$ = `["not","Rained"]`

Horn clause (Rained $\Rightarrow$ Wet) by

$s(\text{Rained} \Rightarrow \text{Wet})$ = `["imp","Rained","Wet"]`

Knowledge Base $\mathcal{KB} = (\neg\text{Rained}) \wedge (\text{Rained} \Rightarrow \text{Wet})$ by

$s(\mathcal{KB})$ =

```
["and", ["not", "Rained"], ["imp", "Rained", "Wet"]]
```

Basis encoding decompositions

The recursive structure of the nested lists `expression` $=s(f)$ is exploited in finding tensor network representations of f.

```
from tnreason.representation import formula_to_cores
↪  as ftc
coreDict =
↪  ftc.create_formula_computation_cores(expression)
```

The function creates a tensor networks of basis encodings to connectives by:

- If $s(f)$ a list, add $\beta^{s(f)[0]}$ and add tensors from recursive calls to the subformulas $s(f)[1:]$
- Return empty dictionary, when $s(f)$ is a string (then an atom)